Holarchy on Paper, Hierarchy in Code: An Industrial Experience Report on a Koestler-Inspired Multi-Agent System for Ticket-to-Pull-Request Automation

Luis Felipe Soares Coelho
Independent Researcher, Brazil
felipecoelho.ness@gmail.com

***Abstract*—Autonomous coding agents are moving from single sessions toward systems of agents that react to tickets and events and return pull requests (PRs). A recurring architectural answer is a flat *router/worker* split, recently proposed again by DeepSeek's "super-agent" design. We report on an alternative built and deployed at a Brazilian B2B SaaS company between March and August 2026: a multi-agent system designed as a *holarchy* in the sense of Koestler, where each agent (*holon*) is at once an autonomous whole and a cooperative part. The system takes an issue-tracker ticket, localises code through a seven-layer search, runs fifteen parallel LLM runners with distinct repair strategies, ranks them with deterministic scores, and opens a PR. We describe two generations of the system, a code audit that compares the intended holarchy with what the code actually does, and field data: 22 PRs on 11 tickets in 6 repositories, opened by 4 engineers, of which 2 PRs (one ticket) were merged; plus 17 autonomous investigation PRs and 9 ticket analyses. Two properties of the holarchy survived contact with production: delegated authority over an irreversible action, and an upward channel through which runner behaviour revises the parent's diagnosis. Peer negotiation, refusal and veto did not survive. We distil five lessons, chiefly that correctness proxies, repository selection and PR hygiene dominated outcomes far more than agent topology.**

***Index Terms*—LLM agents, multi-agent systems, holonic systems, automated program repair, experience report**

I. Introduction

LLM-based agents now resolve a meaningful share of benchmark issues [1]–[4], and practitioners are composing them into systems that act on real engineering workflows. The bottleneck is shifting from the model to human attention: a developer can supervise only a handful of agent sessions at once [5]. Architectures that let agents take work from tickets and events without per-task prompting are therefore attractive.

Most such architectures are hierarchical. DeepSeek's super-agent prototype [5] is representative: a router session receives triggers, evaluates them, and launches one executor session per ticket; executors report to the router and have no tool to message one another. Role-play frameworks such as MetaGPT [6] and ChatDev [7] also fix the chain of command.

Manufacturing faced a similar tension three decades ago and answered with *holonic* systems [8]–[10], built on Koestler's notion of the holon [11]: a unit that is self-assertive toward its parts and integrative toward its whole (the "Janus effect"). A holarchy is a nesting of such units, in which authority is distributed rather than concentrated at the top.

This paper reports on applying that idea to ticket-to-PR automation. The author designed and built the system as an individual engineer. It was first prototyped publicly in March 2026 [12], [13], evolved through two generations, and was used on production repositories. Our contributions are:

- a holarchic architecture for coding agents and its mapping onto a concrete pipeline (Section III);
- a code-level audit that separates what was implemented from what was only designed, an analysis rarely reported for agent systems (Section IV);
- field data from real use, including failures (Section V); and
- lessons for practitioners, contrasted with the router/worker alternative (Section VI).

II. Background and Related Work

Holonic systems. PROSA [8] defined product, resource and order holons for manufacturing control; holonic multi-agent systems [9], [10] generalised the idea to software agents that can merge into super-holons and negotiate. The closest recent work is Ashfaq et al. [14], which proposes LLM reasoning inside supervisor, planner and task holons for systems of systems. That proposal is conceptual and targets smart-city mobility. To our knowledge no prior report describes an implemented, field-used holarchic architecture for software engineering.

LLM agents for software engineering. Single-agent systems [2], [3], [15] and fixed pipelines [4] dominate issue resolution. Multi-agent frameworks [6], [7], [16] coordinate roles through conversation; surveys catalogue the space [17], [18]. Our runner layer is closest to repeated sampling with

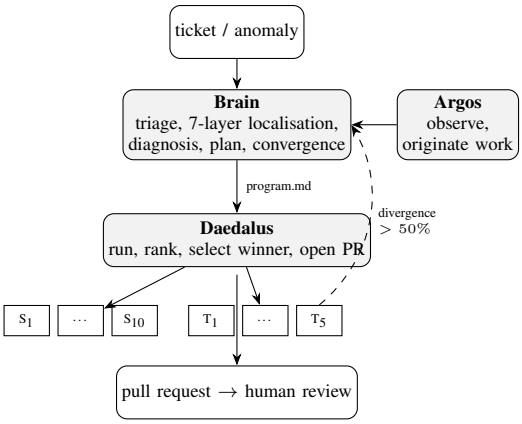


Fig. 1. The holarchy as deployed. Solid arrows: delegation. Dashed arrow: the upward channel by which thinker runners revise Brain's diagnosis. S: snipers, T: thinkers.

TABLE I
HOLARCHIC PROPERTIES, INTENDED VS. REACHABLE IN CODE (IMPL.: IMPLEMENTED; +T: COVERED BY TESTS)

Property	Status	Evidence
Delegated irreversible action (winner, PR)	impl.	Daedalus opens PR before Brain validates
Upward revision of parent belief	impl.+T	divergence > 0.5, $\leq$3 rounds
Autonomous origination of work	impl.	Argos scout $\rightarrow$ intake (gen. 2)
Parent veto with reformulation	impl.+T	1 counter round (gen. 2)
Subordinate refusal / counter-proposal	design	statuses declared, never produced
Peer messaging between runners	absent	runners isolated by design
Asynchronous holon integration	absent	blocking HTTP, up to 15 min

a verifier [19], [20], but with heterogeneous strategies and a feedback edge into diagnosis.

III. Design: A Holarchy for Ticket-to-PR

Holons. The system has three holons (Fig. 1). *Brain* owns understanding: it watches the tracker, triages the ticket (bug, false positive, needs information), selects the repository, localises code, diagnoses a root cause and writes a plan. *Daedalus* owns change: it turns the plan into competing patches and decides which one ships. *Argos* owns observation: it polls monitoring and tracker sources and can originate work. Each holon is a separate process with its own API and store. The design intent, stated in the system's documentation, was that holons "propose, evaluate and decide together".

Localisation. Brain extracts search terms from the ticket in two languages and several identifier styles, then runs seven layers: exact `git grep`, cross-term regex, path filtering, an LLM pass over the tree, recently modified files, test-to-source inference, and an LLM pass over entry points. The two LLM layers run only when fewer than three files have been found. Files are ranked by how many layers hit them.

Competing runners. Daedalus runs ten *sniper* runners in git worktrees, each with a narrow strategy (inline fix, guard, validation, condition fix, fallback, import, async, extraction, type check, simplification). It also runs five *thinker* runners in separate processes with broader strategies (fix with tests, module refactor, architectural fix, comprehensive fix, defensive rewrite) and a larger context window (25k vs. 12k characters). Each runner scores itself with a deterministic rubric worth 100 points: tests passing, diff size, absence of unsafe type casts, no new dependencies, import hygiene, touching a target file, and lint-like checks. The final rank is $0.5\, s_{\text{self}} + 0.3 \min(5\, n_{\text{pass}}, 100) + 20\, [\Delta > 0]$.

Upward correction. After each round, Brain compares the files the thinker runners edited with its own predicted file set. If more than half fall outside the prediction, Brain treats its hypothesis as falsified, merges both views into a revised diagnosis and re-delegates, for at most three rounds. A subordinate's behaviour thus changes the superordinate's belief, which is the integrative face of the holon made operational.

Delegated authority. Daedalus, not Brain, selects the winner, pushes the branch and opens the PR. Brain's later LLM validation annotates the result but cannot retract it. The part holds authority over an irreversible action, which a pure worker does not.

Second generation. *Nous* (May–August 2026) placed a conversational meta-holon above Brain and moved Argos to the front of an autonomous chain: a monitoring scout detects a slow endpoint, Nous opens a ticket, Brain writes an investigation PR, and optionally a fix is drafted and debated by five LLM personas before a human decides. Nous can veto a Brain plan that violates the user's stated hard preferences, and request one reformulation.

Implementation. The first generation comprises 287 commits (20 Mar to 7 Apr 2026) in TypeScript with an append-only Postgres event store and a CQRS projection for graph memory; the second has 353 commits and about 46k lines of TypeScript plus a small Python service. The three public prototypes [13] carry 970 passing tests at the time of writing. All runners used one mid-tier commercial model (Claude Sonnet 4).

IV. Design Versus Code: An Audit

Agent papers usually describe intended architectures. We audited both generations against their source in November 2026, labelling each claim as implemented-and-tested, implemented, or design-only. Table I summarises the holarchic properties.

Three findings matter beyond this system. First, *holarchy eroded toward hierarchy under delivery pressure*: the production path became a fixed sequential script, while the richer agent loop (a 15-state machine and 29 tools) was never wired to the server. Negotiation endpoints exist but always accept. Second, *documentation overstated the code*: the documented scoring rubric is dead code, the "thinkers" were described as a stronger model but ran the same one, and the state count differed. Third, *contracts drifted between holons*: in

TABLE II
FIELD DATA, MARCH–MAY 2026

Generation 1: ticket-to-PR	
PRs opened	22
distinct tickets (incl. 1 test run)	11
repositories	6
engineers who ran it (PR authors)	4
by month (Mar / Apr / May)	16 / 1 / 5
PRs merged	2 (1 ticket, 2 branches)
PRs closed without merge	19
PRs still open	1
Tickets with an automated analysis comment	9
Generation 2: monitoring-to-investigation	
Investigation PRs (one day)	17, none merged
PRs with a five-persona debate verdict	3
Early live test (4 tickets, 20 Mar)	
Triage / localisation succeeded	4/4, 4/4
PRs produced	0/4 (edit-format parsing)
Mean wall time per ticket	36.4 s

the second generation the chat path's delegation payload no longer matched Daedalus's schema and was rejected with HTTP 400; cost and time cut-offs existed but were disabled on the main loop. Seven corrective commits were needed in November 2026 to restore the multi-runner path. Independent processes with separate contracts make such drift easy and silent; contract tests between holons would have caught it.

V. FIELD USE

Method. The PR body generated by Daedalus contains a distinctive section (a per-runner analysis table). We searched the company's GitHub organisation for it, then read state, target branch and closing comments of every hit. We did the same for the ticket-analysis comment, the investigation-PR branch prefix and the debate marker of the second generation. The company and its repositories are anonymised.

Adoption. Table II shows that three engineers other than the author ran the system on their own tickets (9 of the 22 PRs), in March and again in May, without any mandate to do so. Two engineers reported also using the system's review mode to draft code-review comments. We could not verify this from artefacts: that mode deliberately produces plain prose without markers, and the two engineers' 717 inline review comments in the period carry no machine fingerprint. We therefore treat it as self-reported.

Outcomes. One of ten real tickets reached production through the system's own PRs, merged on both the release and development branches after about ten days. Closing comments on the other PRs reveal the failure modes:

- *Duplicate PRs per ticket.* Each convergence round that found a winner opened its own PR; one ticket accumulated six PRs and another three. Many of the 19 closures are such duplicates or later backlog clean-ups after two months without activity.
- *Wrong repository.* One ticket was localised in a sibling repository and patched there.
- *Stale base.* One PR was cut from an outdated branch and carried 103 unrelated files.
- *Misdiagnosis.* In the second generation, all 17 investigation PRs were closed; the one we examined in depth blamed a Markdown documentation file because the endpoint's code lived in another repository.

No closing comment cited a defect in the generated code itself. The evidence is too thin to claim code quality, but it does show where the system lost: before and after code generation, not inside it.

Cost. The only fully instrumented run (a synthetic ticket through the agent loop) consumed 481k input and 14k output tokens over 20 turns, about US$2.76. The deterministic pipeline used in the field is cheaper per call but runs fifteen runners per round.

VI. LESSONS

L1: Keep the two holonic properties that cost nothing. Delegated authority and upward correction were cheap to build and survived every refactoring. Negotiation protocols were expensive and were the first thing cut. A holarchy for agents should start from these two.

L2: Topology mattered less than the edges of the pipeline. Repository selection, base-branch freshness and PR deduplication caused most losses. Router/worker designs [5] face the same edges; the holarchy did not remove them.

L3: Deterministic scores are not correctness. Our rubric rewarded small, typed, dependency-free diffs, and, in repositories without tests, ranking fell back to size and style alone. Repeated sampling only pays when the verifier measures the property of interest [19], [20].

L4: Audit the design you publish. Every overstatement we found (model, rubric, states, negotiation) was in documentation written during a competition deadline. A reachability audit took less time than writing the original documentation.

L5: Measure adoption, not only accuracy. A 1-in-10 merge rate looks poor, yet engineers kept choosing to run the system. Field reports should record who used an agent and for what, alongside whether its PRs were merged.

Threats to validity. The data come from one company, two generations and a small number of tickets. Fingerprint search finds only artefacts that kept the generated text; PRs that engineers rewrote by hand are invisible, so the merge rate is a lower bound on influence. The author built the system and conducted the audit.

VII. CONCLUSION

A Koestler-style holarchy is a useful lens for agent systems: it names the two properties (authority delegated to parts, information flowing up to revise the whole) that distinguish a holon from a worker. In our deployment those two survived and the rest eroded into a conventional pipeline. DeepSeek's recent super-agent [5] keeps the router in charge of every decision; our experience suggests that letting the part decide, and letting it correct the whole, is feasible in production today. Our next step is a controlled evaluation on resolved tickets with known human fixes, using the repaired multi-runner path.

REFERENCES

[1] C. E. Jimenez, J. Yang, A. Wettig *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?" in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2310.06770.

[2] J. Yang, C. E. Jimenez, A. Wettig *et al.*, "SWE-agent: Agent-computer interfaces enable automated software engineering," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024, arXiv:2405.15793.

[3] X. Wang, B. Li, Y. Song *et al.*, "OpenHands: An open platform for AI software developers as generalist agents," in *International Conference on Learning Representations (ICLR)*, 2025, arXiv:2407.16741.

[4] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang, "Agentless: Demystifying LLM-based software engineering agents," 2024, arXiv:2407.01489.

[5] DeepSeek Harness Team, "Exploring super-agent design: Towards a more autonomous agent system," Oct. 2026, technical post, 31 Oct 2026. Prototype: https://github.com/deepseek-ai/dsh-superagent-prototype.

[6] S. Hong, M. Zhuge, J. Chen *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024, arXiv:2308.00352.

[7] C. Qian, W. Liu, H. Liu *et al.*, "ChatDev: Communicative agents for software development," in *Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024, arXiv:2307.07924.

[8] H. Van Brussel, J. Wyns, P. Valckenaers, L. Bongaerts, and P. Peeters, "Reference architecture for holonic manufacturing systems: PROSA," *Computers in Industry*, vol. 37, no. 3, pp. 255–274, 1998.

[9] C. Gerber, J. Siekmann, and G. Vierke, "Holonic multi-agent systems," German Research Center for Artificial Intelligence (DFKI), Tech. Rep. RR-99-03, 1999.

[10] A. Giret and V. Botti, "Holons and agents," *Journal of Intelligent Manufacturing*, vol. 15, no. 5, pp. 645–659, 2004.

[11] A. Koestler, *The Ghost in the Machine*. London: Hutchinson, 1967.

[12] L. F. S. Coelho, "The whole and the part: Holonomic software architecture," 2026, open-source book, first public release 8 Mar 2026. https://github.com/Felipeness/the-whole-and-the-part.

[13] ——, "Holonomic agent prototypes: claude-holonomic, holon-daedalus, holon-argos," 2026, commits from 17–18 Mar 2026. https://github.com/Felipeness/claude-holonomic, https://github.com/Felipeness/holon-daedalus, https://github.com/Felipeness/holon-argos.

[14] M. Ashfaq, A. R. Sadik, T. Mikkonen, M. Waseem, and N. Mäkitalo, "LLM-enhanced holonic architecture for ad-hoc scalable SoS," 2025, arXiv:2501.07992.

[15] Y. Zhang, H. Ruan, Z. Fan, and A. Roychoudhury, "AutoCodeRover: Autonomous program improvement," in *ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, 2024, arXiv:2404.05427.

[16] Q. Wu, G. Bansal, J. Zhang *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," 2023, arXiv:2308.08155.

[17] J. He, C. Treude, and D. Lo, "LLM-based multi-agent systems for software engineering: Literature review, vision and the road ahead," 2024, arXiv:2404.04834.

[18] J. Liu, K. Wang, Y. Chen *et al.*, "Large language model-based agents for software engineering: A survey," 2024, arXiv:2409.02977.

[19] B. Brown, J. Juravsky, R. Ehrlich *et al.*, "Large language monkeys: Scaling inference compute with repeated sampling," 2024, arXiv:2407.21787.

[20] B. Chen, F. Zhang, A. Nguyen *et al.*, "CodeT: Code generation with generated tests," in *International Conference on Learning Representations (ICLR)*, 2023, arXiv:2207.10397.